Split the Circuit

Building a Distributed Quantum Compiler with Qiskit and GenAI

BITS Pilani, K K Birla Goa Campus

IBM Qiskit Fall Fest

Advanced Problem Statement
72-hour hackathon

The challenge

Real quantum processors have limited qubit capacity. One possible route to larger computations is to connect several smaller quantum processing units (QPUs) using shared entanglement (e-bits) and classical communication.

Your task is to build a software stack that makes such a distributed machine usable in simulation: a compiler that partitions a Qiskit circuit across QPUs, a scheduler that orders quantum and classical operations in time, and a controller that executes the schedule, manages communication between QPUs, and reconstructs the result. Handle operations between QPUs using mid-circuit measurements and teleportation, and use circuit cutting where appropriate. Use an LLM or another GenAI tool to find cheaper partitions or execution plans. Finally, demonstrate that the distributed result matches the original circuit using Qiskit Aer.

The challenge is to optimise the trade-off between e-bit consumption, circuit-cutting sampling overhead, classical communication, and execution time. A valid solution must respect the capacity and communication constraints of the simulated QPU network.

1 Basic definitions

QPU	One small quantum processor. A distributed machine links several QPUs.
Non-local gate	A two-qubit gate whose qubits are assigned to different QPUs and therefore cannot run directly as a local gate.
E-bit	A shared entangled Bell pair between two QPUs. It is a resource consumed by teleportation.
Quantum communication	Generation and consumption of e-bits between QPUs.
Classical communication	Sending measurement outcomes between QPUs so that the receiver can apply the required correction.
State teleportation	Moves a qubit's state to another QPU using an e-bit, measurement, and classical feedforward.
Gate teleportation	Implements a supported non-local gate using shared entanglement and classical feedforward without simply moving the data qubit.
Gate packing	Groups compatible controlled gates so that an e-bit can be reused where the protocol permits.
Mid-circuit measurement	Measuring a qubit before the circuit ends, then continuing execution.
Dynamic circuit	A circuit using mid-circuit measurements and classical logic to choose later operations. In Qiskit, this can use `if_test`.
Circuit cutting	Splits a circuit into independently executed pieces and recombines their results classically.
Sampling overhead	Extra shots required to maintain accuracy after cutting. It can grow rapidly as cuts accumulate.
Qubit reuse	Measuring and resetting a qubit mid-circuit so that the same physical qubit can be used later.
Stitching	Combining distributed measurement results or fragment estimates into a result for the original circuit.
Makespan	Total modelled time from the start to the completion of distributed execution.

2 Compiler and partitioning

The compiler takes as input a `QuantumCircuit` (or QASM file) and a network configuration. The network configuration must specify the number of QPUs, data-qubit capacity per QPU, communication-qubit capacity per QPU, link connectivity, and e-bit generation rate for each link. It must also provide the operation durations used by the scheduler.

1. Construct the qubit-interaction graph of the input circuit. Vertices represent logical qubits, and weighted edges represent interactions between them. State how the edge weights are defined.
2. Implement a static graph-partitioning baseline, such as min-cut or Kernighan–Lin. Assign logical qubits to QPUs while respecting their capacities.
3. Extend the assignment to a time-sliced partition: logical qubits may be assigned to different QPUs at different circuit layers. Include the cost of moving a quantum state or otherwise handling a change of assignment. Reassignment must not be treated as free.
4. For every operation whose operands are assigned to different QPUs, select a valid coverage strategy. The available strategies should include state teleportation, gate teleportation (including gate packing where supported), and circuit cutting.
5. Generate the local circuit fragments, the layer-wise logical-qubit-to-QPU mapping, and a communication plan describing every non-local operation and the strategy selected for it.

Output. A static baseline partition, an optimised time-sliced partition, local circuit fragments, and a complete communication plan.

3 Non-local operations

3.1 Teleportation

Implement state teleportation and at least one gate-teleportation strategy. Teleportation uses e-bits, mid-circuit measurements, and classical feedforward. The receiving QPU must wait for the relevant measurement outcomes before applying conditional corrections.

For gate packing, specify which gates can share an e-bit and under what conditions. Verify that the resulting construction implements the intended operation.

Output. Working teleportation primitives, resource counts, measurement dependencies, and the classical corrections required by each operation.

3.2 Circuit cutting

Implement at least one circuit-cutting method: wire cutting or gate cutting. Generate the required circuit fragments, execute the fragment variants, and reconstruct a specified global quantity, such as an expectation value or an output probability distribution.

Decompose the circuit into the gate set required by the selected cutting method before applying cuts. Report the number of cuts, fragment variants, total shots, and sampling overhead. The reconstruction must include the signed coefficients or other weights required by the chosen method.

Output. Independently executable fragments, reconstruction procedure, and a comparison of the reconstructed result with the corresponding result from the original circuit.

3.3 Qubit reuse

Qubit reuse is an extension. If implemented, measure and reset qubits only when the circuit's dependencies and state requirements permit reuse. Report the reduction in physical-qubit requirements and verify that the transformed circuit preserves the target result.

4 Communication manager and scheduler

The communication manager tracks e-bit generation, availability, and consumption, together with the creation and delivery of classical messages.

Each link generates e-bits at a specified rate. E-bit generation is slower than a local two-qubit gate. Each QPU has a limited number of communication qubits that hold its half of an e-bit. The scheduler must not use an e-bit before it exists or exceed the communication qubit capacity.

Every teleportation produces measurement outcomes that must reach the receiving QPU before the conditional correction can run. Model this as a classical message with a specified latency. The scheduler must account for this waiting time, as well as measurement and reset durations.

The scheduler must order:

- local one- and two-qubit gates;
- e-bit generation and allocation;
- mid-circuit measurements and resets;
- classical-message transmission and delivery;
- conditional corrections and subsequent dependent gates.

Operations may run in parallel only when their dependencies and resource requirements permit it. The scheduler must respect QPU capacities, communication-qubit limits, link generation rates, and message dependencies.

Output. An execution timeline, resource-occupancy information, and an estimated makespan.

5 Controller and reconstruction

The controller executes the scheduled plan using Qiskit Aer. It dispatches operations to the simulated QPUs, routes classical messages, applies conditional corrections, and collects the results.

There are two distinct execution models:

- **Teleportation-based distribution.** The QPUs remain quantum-correlated through shared e-bits. A faithful Aer simulation may represent the network as one global dynamic circuit with QPU-local registers and explicit communication operations. Retain the logical qubit mapping throughout execution.
- **Circuit-cutting distribution.** Circuit fragments are executed independently. Their results are combined using classical post-processing. Retain fragment, measurement-basis, preparation, and reconstruction metadata as required by the chosen cutting method.

Do not treat these as the same execution or reconstruction procedure. For teleportation, map final measurement bits back to the original logical-qubit order. For circuit cutting, reconstruct the selected global quantity and report its uncertainty.

Output. Execution records, per-QPU or per-fragment results, and the reconstructed logical output or observable estimates.

6 Optimisation

The compiler minimises a weighted cost subject to hard constraints:

$$C = w_1 N_{\text{ebit}} + w_2 O_{\text{sampling}} + w_3 T_{\text{makespan}} + w_4 N_{\text{messages}}.$$

Here, N_{ebit} is the number of e-bits consumed, O_{sampling} is the extra sampling overhead introduced by circuit cutting relative to the uncut circuit, T_{makespan} is the estimated execution time, and N_{messages} is the number of classical messages.

Teams must state the weights used in their experiments. If the organisers provide default weights and operation durations, use those values for official comparisons.

The hard constraints are:

1. QPU data-qubit and total physical-qubit capacities must not be exceeded.
2. The number of occupied communication qubits must not exceed the configured capacity.
3. An e-bit cannot be consumed before it has been generated.
4. A conditional operation cannot execute before its required classical message arrives.
5. The distributed result must match the original circuit within the specified tolerance.

Compare the optimised compiler with the static graph-partitioning baseline. The comparison must use the same circuits, network configuration, operation durations, and cost weights.

7 GenAI component

The LLM or other GenAI tool must contribute meaningfully to at least one of the following:

- proposing or improving a static or time-sliced qubit assignment;
- choosing the coverage strategy for non-local operations;
- proposing or improving a schedule or resource-allocation plan.

One possible approach is to summarise the circuit as a layer-wise interaction list and ask the model to propose an assignment. Another is to use an agent that proposes a partition, calls a cost evaluator and correctness validator, reads the feedback, and iterates.

Every proposal must be converted into a structured representation and checked. Reject invalid assignments, capacity violations, impossible schedules, and non-equivalent circuit transformations. The write-up must state exactly what the GenAI tool contributed.

Ablation. Compare the optimisation workflow with and without GenAI. Use the same benchmark instances, network configuration, cost weights, and evaluation procedure. Report whether GenAI improves solution quality, search time, or another clearly defined measure. Do not attribute the work of a deterministic optimiser to the LLM.

8 Benchmarks and verification

8.1 Circuit families and scale

Evaluate the compiler on the following benchmark families:

- Quantum Fourier Transform (QFT);
- Quantum Volume-style circuits;
- QAOA on small random graphs;
- random controlled-phase fraction circuits.

Use circuits of approximately 8–14 logical qubits, distributed over 2–4 QPUs of approximately 3–6 data qubits each. Include one or more larger circuits for scaling plots where resources permit. The implementation must accept circuit and network configuration inputs in a documented format; benchmark-specific hard-coded partitions are not sufficient.

8.2 Correctness

Run the original and distributed circuits using Qiskit Aer. For small circuits, compare statevectors or state fidelity where the execution model permits. For shot-based outputs, compare probability distributions using total variation distance (TVD) or another justified statistical metric.

For circuit cutting, compare the reconstructed target quantity with the corresponding reference value and report the sampling uncertainty. State the verification metric and tolerance used. A run that violates a hard resource constraint is invalid even if its output matches the reference.

8.3 Performance

Report the following for each benchmark:

- number of e-bits consumed and e-bits per original two-qubit gate;
- circuit-cutting sampling overhead and total shots, where applicable;
- estimated makespan;
- number of classical messages;
- peak data-qubit and communication-qubit occupancy per QPU;
- verification error and whether all hard constraints were satisfied.

Compare the results against the static partitioning baseline. The organisers may use additional benchmark instances for evaluation.

9 Submission

Submit one archive named `team_name.zip`. It must contain:

1. `src/`, `README.md`, and a pinned environment or dependency file;
2. the compiler, scheduler, communication manager, controller, reconstruction, and verification implementation;
3. benchmark circuits or generators, network configurations, and scripts to reproduce the experiments;
4. result files containing correctness and performance metrics;
5. a technical report describing the architecture, algorithms, assumptions, cost weights, limitations, and GenAI ablation;
6. a notebook or scripted walkthrough and a short demonstration.

Include a command to reproduce figures and tables from saved results without submitting new quantum jobs. Identify reused packages and AI assistance. Do not include credentials or API keys.

10 Suggested evaluation

Criterion	Points	Assessment
Correctness	25	Equivalence checks pass; resource constraints are respected; cutting reconstruction is validated where used.
Cost efficiency	25	E-bit use, sampling overhead, and makespan compared with the baseline under common weights.
System design	20	Compiler, scheduler, controller, communication manager, and reconstructor work together; implementation is reusable.
GenAI and ablation	15	GenAI contributes to optimisation; proposals are validated; with/without-GenAI results are compared.
Explanation and demo	15	Clear technical explanation, reproducible experiments, benchmark analysis, and walkthrough.

11 Suggested 72-hour plan

Hours 0–24
Establish the baseline. Implement or extend partitioning, time-sliced assignment, and teleportation primitives. Verify small circuits.

Hours 24–48
Integrate resource-aware communication, scheduling, and runtime control. Implement circuit cutting and reconstruction. Add the GenAI proposal loop and deterministic validation.

Hours 48–72
Run benchmarks, compare with the baseline, perform the GenAI ablation, analyse scaling and failure cases, and prepare the report and demonstration.

12 Simulation environment and limitations

The primary evaluation environment is Qiskit Aer. Entanglement links between independent commercial QPUs are not generally available as a native multi-QPU execution service; therefore, the distributed network is a simulated architecture. Real IBM hardware access is not required.

A successful result is not required to demonstrate quantum advantage. The goal is to build a correct, resource-aware distributed compiler and execution stack, quantify its costs, and evaluate the optimisation methods against a transparent baseline.

References

[1] Burt, Chen and Leung, "Generalised Circuit Partitioning for Distributed Quantum Computing," arXiv:2408.01424.

[2] Sinha, Jha and Raj, "Circuit Partitioning Using Large Language Models for Quantum Compilation and Simulations," arXiv:2505.07711.

[3] Uchehara et al., "GidNET: Graph-Based Identification of Qubit Network for Qubit Reuse," IEEE QCE 2024.

[4] Kole et al., "Dynamic Transformation of Quantum Algorithms for Resource-Constrained Architectures," SN Computer Science, 2026.

[5] Ma and Li, "Understanding and Estimating the Execution Time of Quantum Circuits," ACM TOSEM 2025; arXiv:2411.15631.

[6] IBM Quantum documentation on dynamic circuits, classical feedforward, `if_test`, and mid-circuit measurement.

[7] "Perspectives on Utilization of Measurements in Quantum Algorithms," arXiv:2507.04325.

[8] Harada et al., "Exponential-to-Polynomial Scaling of Measurement Overhead in Circuit Knitting via Quantum Tomography," arXiv:2512.19623.